

ByteBind: Making the Private Network Part of the Login

I started with what felt like a pretty boring problem.

I had a public application with a management interface.

That application also had privileged access to resources on a private overlay network.

So naturally, the management interface needed stronger protection than the rest of the site.

The obvious answer was:

Put the management page on the private network.

And honestly, if that works for your application, you should probably do that.

ByteBind exists for the annoying cases where it doesn't.

Maybe the application needs to stay on its normal public origin. Maybe the management UI shares the same frontend and deployment. Maybe you want ordinary browser behavior, public DNS, or a conventional reverse proxy in front of the app.

But there was one property I still wanted:

You should only be able to manage the application while you are actively present on the private network that the application itself has privileged access to.

Not "you logged in from there six hours ago."

Not "your account once belonged to an administrator."

Not "your source IP looks vaguely internal."

Right now.

And preferably without asking the user to type anything.

No username.

No password.

No WebAuthn prompt.

No "check your phone."

If you are on an authorized device, management should simply work.

If you are not, it should not.

That eventually became ByteBind.

This is not "VPN = trusted"

There is an obvious objection here:

Aren't we supposed to be moving away from "they're on the VPN, therefore they're trusted"?

Yes.

ByteBind does not make that assumption.

Being able to send a packet from the private network is not sufficient.

A ByteBind authorization combines:

```
fresh transaction
+ live private-network reachability
+ cryptographically authenticated device identity
+ explicit authorization policy
+ browser transaction continuity
+ single-use redemption
```

The private network is not treated as a magic trusted perimeter.

It is used as four things at once:

1. an authenticated transport;
2. a device identity source;
3. a policy source;
4. and a path the browser must demonstrably traverse during the authentication ceremony.

That distinction matters.

The question is not:

"Are you inside?"

It is:

"Can this browser, right now, participate through an explicitly authorized identity on the same private trust fabric whose resources this application can control?"

That is a much more useful security statement.

The weird realization

The thing that unlocked the design was realizing that the private network did not need to carry the entire management request.

It only needed to participate in authenticating it.

That gave me three roles:

```
Client
Server
Provider
```

The Client is the browser.

The Server is the public application.

The Provider is the private-network authority that can answer questions like:

- what device is this;
- is it actually live on the private network;
- is it authorized;
- what claims, if any, should the application learn?

The Server does not need raw access to the private network's identity system.

The Provider does not need to serve the application.

The browser is the thing that proves it can participate in both worlds.

That led to a ten-message ceremony.

PLEASE, BEGIN, TRY, WHO, PROVE, ATTEST, AFFIRM, REDEEM, GRANT, RESPONSE

Yes, I named the messages like verbs.

They are not HTTP methods. They are logical protocol messages.

The flow is:

```
Client      Server      Provider
|           |           |
|----- PLEASE ---->|           |
|           |----- BEGIN ---->|
|           |<----- TRY -----|
|<----- WHO -----|           |
|----- PROVE ----->|
|<----- ATTEST -----|
|----- AFFIRM ---->|           |
|           |----- REDEEM ---->|
|           |<----- GRANT -----|
|<----- RESPONSE ---|           |
```

Or, flattened:

```
Client -> Server  PLEASE
Server -> Provider BEGIN
Provider -> Server TRY
Server -> Client  WHO
Client -> Provider PROVE
Provider -> Client ATTEST
Client -> Server  AFFIRM
Server -> Provider REDEEM
Provider -> Server GRANT
Server -> Client  RESPONSE
```

And, somewhat delightfully, the whole thing reads like a conversation:

*PLEASE do this.
BEGIN a transaction.
TRY proving them.
WHO are you?
PROVE who I am.
I ATTEST you are you.
I AFFIRM I am me.
REDEEM this ticket.
I GRANT you authority.
Here is your RESPONSE.*

That is not the formal definition of the protocol, obviously.
But it is a surprisingly good mnemonic for the state machine.

PLEASE

The Client asks the Server to perform a privileged operation.
For a transaction-bound request, this is the actual operation being requested.
For example:

```
PLEASE restart service X
```

The Server does not execute it yet.

BEGIN

The Server asks the Provider to create a fresh ByteBind transaction.
The Provider binds that transaction to things like:

- the relying application;
- the allowed browser origin;
- the requested policy;
- the audience;
- and, for transaction-bound authorization, the exact request.

TRY

The Provider returns fresh transaction material to the Server.
This includes a challenge identifier and client-visible challenge material.

WHO

The Server passes the client-visible portion back to the browser.
Conceptually:

"Okay. Who are you? Prove that you are on an authorized device."

PROVE

Now the browser contacts the Provider directly over the private network.

That private route is important.

If the browser is no longer attached to the private network, this message does not succeed.

The Provider also identifies the connecting device through the private network's own authenticated identity mechanism.

With Tailscale, that can be node identity and tags.

Another private overlay could supply its own authenticated device identity in the same role.

The exact Provider is replaceable.

The important part is that the identity comes from the authenticated private trust fabric itself.

ATTEST

If the Provider accepts the device and policy, it responds directly to the browser over the private path.

It is effectively saying:

"I attest that the device I just observed is the authorized identity participating in this transaction."

This proves the browser can not only send into the private network but receive the corresponding attestation response.

The response also contains fresh secret material that the browser needs to continue.

AFFIRM

The browser carries the completed proof back to the public Server.

It is now saying:

"I affirm that I am the participant that received that attestation."

The browser has demonstrated continuity across both channels:

```
public HTTPS
+
private authenticated path
```

But importantly, the Server still does not execute the protected operation.

AFFIRM is proof.

It is not authority.

REDEEM

The Server sends the completed transaction to the Provider:

"Redeem this ticket."

The Provider verifies that the proof is valid, fresh, scoped to this Server and operation, and has not already been consumed.

Redemption is single-use.

GRANT

Only after successful redemption does the Provider return the actual authorization result:

"I grant you authority to perform the operation covered by this transaction."

That distinction ended up mattering more than I expected.

The Client does not grant itself authority.

The Server does not infer authority merely because it received a plausible-looking proof.

The Provider explicitly converts the successfully completed, one-time transaction into a scoped grant.

RESPONSE

Only now does the Server exercise that authority.

It performs the protected operation and returns the original application response:

```
RESPONSE: service restarted
```

The protocol therefore does not have to be:

```
authenticate
then
do the thing
```

It can wrap the thing itself.

Binding authorization to the actual request

For particularly sensitive operations, ByteBind can bind the attestation to the exact request rather than merely creating a short-lived login session.

The browser computes a digest of the request:

```
Q = SHA-256(  
  method  
  || target  
  || selected headers  
  || body  
)
```

using deterministic encoding and length-prefixing.

That digest becomes part of the cryptographic transcript.

So the proof does not merely mean:

"An authorized device authenticated."

It means:

"An authorized device participated in approval of this exact request."

And **GRANT** means:

"The Provider has authorized the Server to exercise the authority associated with this exact redeemed transaction."

That is a much nicer property for management operations such as:

```
restart this service  
rotate this credential  
publish this release  
delete this resource
```

The transaction can be single-use, and the operation can be executed at most once.

For lower-risk operations, the same ceremony can instead mint a short-lived management lease.

Ambient authentication

The session version of ByteBind is intended to be boring for the person using it.

Open the management page.

The browser performs the ceremony silently.

If the current device is authorized and attached to the private network, the management session appears.

While the page remains open, the browser can periodically re-attest.

For example:

```
attestation window: 30 seconds
redeem window:      10 seconds
session lease:       3 minutes
renewal interval:    1 minute
```

If the device leaves the private network, loses its authorization, or simply stops being able to reach the Provider, renewals stop succeeding.

The session then expires naturally.

There is no permanent statement that says:

"This browser is trusted."

There is only:

"This browser demonstrated authorized private-network participation recently enough that the current lease is still valid."

Why not just give the public app access to the identity system?

Because it does not need it.

One of the cleaner parts of the architecture is that the Provider acts as a privacy and policy boundary.

It might know:

```
private IP
stable node ID
device tags
user identity
shared-node status
device posture
```

The relying application might only need:

```
active = true
authorization = manage:read
```

So that is all it should receive.

A public application should not automatically become a privileged observer of the entire private network merely because it wants to authenticate an administrator.

Tailscale is one provider, not the protocol

ByteBind actually started life under the name **TailBind**.

That made sense at first. The problem was Tailscale-shaped, the first provider was Tailscale, and the design grew out of asking how a public application could require live participation from an authorized tailnet device.

Then the protocol kept getting more general.

What the Provider really needs is roughly:

1. a private path that public internet clients cannot directly use
2. cryptographically established device or endpoint identity
3. an authoritative way to map the live connection to that identity
4. a policy system capable of deciding whether that identity is allowed

Tailscale gives you that through its authenticated WireGuard overlay and node identity.

Another private overlay could do the same, as long as it meets all four requirements, especially the third: the Provider must be able to tie the live connection it sees to a specific device.

At that point, naming the protocol after one possible Provider stopped making much sense.

So drafts 0.1 through 0.5 were **TailBind**.

Draft 0.6 became **ByteBind**.

The more general idea was never really:

"authenticate using Tailscale."

It was:

use an authenticated private trust fabric as an active participant in a public authentication ceremony.

It just took me a few drafts to notice.

What ByteBind does not claim

A few things are deliberately outside the scope.

ByteBind does not prove which human is physically touching the keyboard.

If an authorized device is compromised, ByteBind is not going to magically fix that.

If you need explicit human presence or phishing-resistant user authentication, WebAuthn or another user authentication mechanism can be layered on top.

ByteBind is also not a new cryptographic primitive.

It uses boring things like:

HMAC-SHA256
HKDF-SHA256
AES-256-GCM
random nonces
single-use state transitions

The interesting part is the ceremony and the trust model, not inventing a new cipher.

That is very intentional.

So... did I accidentally invent a protocol?

Maybe.

The broad family of ideas certainly has prior art.

Multi-channel authentication exists.

Channel binding exists.

Device attestation exists.

Continuous access evaluation exists.

Private network identity exists.

But I have not found an existing standard protocol that combines them in quite this way:

a public relying application delegates live private-network attestation to an independent Provider, the browser must actively traverse both trust domains during the same transaction, and the resulting proof can be bound to either a short-lived session or the exact privileged request being performed.

That does not mean nobody has ever built something similar.

Authentication literature has some wonderfully obscure corners.

So I am being deliberately conservative about novelty.

What I am comfortable saying is:

ByteBind is a private-overlay-backed, liveness-bound authentication protocol built from established cryptographic primitives.

The current specification is still a draft.

I am also seriously considering turning it into an IETF Internet-Draft and asking DISPATCH where the work belongs.

Which is not where I expected to end up when I started with:

"How do I protect the management page for this server?"

But, apparently, here we are.

Yet again, a problem created by **Bad Decisions** resulted in a **Good Decision**.

I may have to rename that project.

ByteBind

© 2026 Bytes & Coffee Digital Studio